Module 11 — Dynamic Programming

Sources: CLRS 4e ch. 14 (Dynamic Programming) · Skiena 3e ch. 10 (Dynamic Programming)

Big Idea

Optimization problems demand **proof that you return the best possible solution.** Skiena frames the whole chapter around the tension this creates:

- **Greedy** algorithms make the best local decision at each step. Fast, but usually **no global-optimality guarantee**.
- **Exhaustive search** tries all possibilities. Always optimal, but at a **prohibitive time cost**.

> *"Dynamic programming combines the best of both worlds. It gives us a way to design custom algorithms that systematically search all possibilities (thus guaranteeing correctness) while storing intermediate results to avoid recomputing (thus providing efficiency)."*

That is the one-sentence definition worth memorizing: **DP is exhaustive search made efficient by never solving the same subproblem twice.**

CLRS says the same thing from the other side. Divide-and-conquer partitions a problem into **disjoint** subproblems; dynamic programming applies when the subproblems **overlap** — when subproblems share subsubproblems. In that regime a divide-and-conquer algorithm does exponentially more work than necessary, re-deriving the same answers. DP solves each subsubproblem **once** and saves the answer. It is a **time–memory trade-off**.

Two more framings that pay off:

- **Skiena's:** *"Dynamic programming is a technique for efficiently implementing a recursive algorithm by storing partial results… Dynamic programming starts with a recursive algorithm or definition. Only after we have a correct recursive algorithm can we worry about speeding it up by using a results matrix."* **Recurrence first, table second. Always.**

- **CLRS's:** the word "programming" here means **a tabular method**, not writing code — the same sense as in "linear programming". Bellman began the systematic study in 1955.

And the honest warning, from Skiena: *"After you understand it, dynamic programming is probably the easiest algorithm design technique to apply in practice. In fact, I find that dynamic programming algorithms are often easier to reinvent than to try to look up. That said, until you understand dynamic programming, it seems like magic. You have to figure out the trick before you can use it."*

The trick, stated plainly: *ask what information about the first* `n−1` *elements would let you decide what to do with the* `n`*-th.* That question generates the state. Everything else is bookkeeping.

Remember months later: *find the recurrence; count its distinct parameter values; pick an evaluation order. If the count is a small polynomial, you have an efficient algorithm. DP lives on objects with an inherent left-to-right order — strings, sequences, rooted trees, polygons — and dies without one.*

What You Should Be Able To Do After This Chapter

- Turn "solve this optimization problem" into a recurrence by asking what state suffices to extend a partial solution.
- Recite both design procedures (CLRS's four steps, Skiena's three) and use them interchangeably.
- Prove optimal substructure with a **cut-and-paste** argument, and recognize when it **fails** (longest simple path).
- Distinguish **overlapping subproblems** from **independent subproblems** — and explain why DP needs both.
- Convert freely between top-down memoization and bottom-up tabulation, and say when each is preferable.
- Derive the running time as *(number of subproblems) × (choices per subproblem)*, or equivalently from the subproblem graph.
- Reconstruct the actual solution, not just its value, via a parent/choice table.
- Reduce the space to `O(one row)` when only the value is needed, and know why that breaks reconstruction.
- Write from memory: rod cutting, matrix chain, LCS, edit distance, LIS (both `O(n²)` and `O(n lg n)`), subset sum / knapsack, linear partition, CYK, optimal BST, Held-Karp.

- Recognize the **edit-distance skeleton** and re-derive LCS, approximate substring matching, and LIS as special cases.
- Explain why subset sum's `O(nk)` algorithm does not prove `P = NP`.
- Say precisely when DP is the wrong tool.

Part 1 — Caching vs. Computation

The canonical illustration: Fibonacci

`F(n) = F(n−1) + F(n−2)`, `F(0) = 0`, `F(1) = 1`. The recursive translation is immediate — and catastrophic.

Why it's exponential. Since `F(n+1)/F(n) → φ ≈ 1.618`, we have `F(n) > 1.6ⁿ` for large `n`. The recursion tree's leaves are all `0` or `1`, so summing them to reach a number that big requires **at least `1.6ⁿ` leaves**, hence that many procedure calls. Skiena reports his laptop taking **4 minutes 40 seconds to compute `F(50)`** — a number you can compute by hand faster.

Three fixes, in increasing order of sophistication:

VERSION	IDEA	TIME	SPACE
`fibNaive`	pure recursion	`Θ(φⁿ)`	`O(n)` stack
`fibMemo`	**caching / memoization**: check the table before computing	`Θ(n)`	`Θ(n)`
`fibBottomUp`	**explicit evaluation order**, no recursion at all	`Θ(n)`	`Θ(n)`
`fibRolling`	notice only the last two values matter	`Θ(n)`	**`Θ(1)`**

Skiena's picture of *why* memoization works is the one to keep: after caching, the recursion tree "has no meaningful branching, because only the left-side calls do computation. The right-side calls find what they are looking for in the cache and immediately return." `fib_c(k)` is called **at most twice** for each `k`, hence `O(n)`.

> ***Take-Home Lesson (Skiena):*** *"Explicit caching of the results of recursive calls provides most of the benefits of dynamic programming, usually including the same running time as the more elegant full solution. If you prefer doing extra programming to more subtle thinking, I guess you can stop here."*

When caching does NOT help — and this is the sharpest diagnostic in the chapter: *"storing partial results would have done absolutely no good for such recursive algorithms as quicksort, backtracking, and depth-first search because all the recursive calls made in these algorithms have distinct parameter values. It doesn't pay to store something you will use once and never refer to again."*

Caching makes sense only when the space of distinct parameter values is modest. For `fib_c(k)` there are only `O(n)` values — a linear amount of space for an exponential amount of time is an excellent trade.

(CLRS Exercise 14.3-2 makes the same point about merge sort: memoization can't speed up a good divide-and-conquer algorithm, because its subproblems never repeat.)

```
#include <vector>

long long fibNaive(int n) {                          // exponential:
only for tiny n
    if (n < 2) return n;
    return fibNaive(n - 1) + fibNaive(n - 2);
}

long long fibMemo(int n) {                           // top-down with
memoization
    static std::vector<long long> memo;
    if ((int)memo.size() <= n) memo.resize(n + 1, -1);
    if (n < 2) return n;
    if (memo[n] >= 0) return memo[n];
    return memo[n] = fibMemo(n - 1) + fibMemo(n - 2);
}

long long fibBottomUp(int n) {                       // bottom-up,
O(n) space
    if (n < 2) return n;
    std::vector<long long> f(n + 1);
    f[0] = 0; f[1] = 1;
    for (int i = 2; i <= n; ++i) f[i] = f[i - 1] + f[i - 2];
    return f[n];
}

long long fibRolling(int n) {                        // O(1) space:
keep only the window
    if (n == 0) return 0;
```

```
    long long back2 = 0, back1 = 1;
    for (int i = 2; i <= n; ++i) {
        const long long next = back1 + back2;
        back2 = back1;
        back1 = next;
    }
    return back1;
}
```

Verified: all four agree for `n = 0..25`; `F(90) = 2 880 067 194 370 816 120`, the largest that fits in a signed 64-bit integer.

> ### *Outside / Engineering Context*
>
> *`F(n)` can also be computed in `Θ(lg n)` by repeated squaring of `[[1,1],[1,0]]` — an important reminder that DP gives you polynomial, not necessarily optimal. The same trick (matrix exponentiation of a linear recurrence's transition matrix) turns any `O(n·k)` linear-recurrence DP into `O(k³ lg n)`, which is the standard competitive-programming move when `n` is `10¹⁸`.*

The two implementations of the same idea

	TOP-DOWN WITH MEMOIZATION	BOTTOM-UP (TABULATION)
Control flow	natural recursion + a "have I seen this?" check	nested loops in size order
Which subproblems get solved	**only those actually reachable**	**all of them**
Constant factors	recursion + table-maintenance overhead	**better** — no call overhead
Space optimization	hard (whole table must persist)	easy (rolling rows)
Risk	stack depth on deep recursions	must get the evaluation order right
Best when	the subproblem space is sparse, or the order is awkward to state	all subproblems are needed anyway (the usual case)

CLRS's summary: *"if all subproblems must be solved at least once, a bottom-up dynamic-programming algorithm usually outperforms the corresponding top-down memoized algorithm by a constant factor... On the other hand, in certain situations, some of the subproblems in the subproblem space might not need to be solved at all. In that case, the memoized solution has the advantage of solving only those subproblems that are definitely required."*

Practical advice for interviews: *write the memoized version first* — it is a one-line change from the recurrence you just derived, so it's much harder to get wrong. Convert to bottom-up only if you need the constant factor or the space optimization.

The subproblem graph

Build a directed graph with **one vertex per distinct subproblem** and an edge `x → y` when solving `x` directly requires `y`. It is the recursion tree with all identical nodes collapsed into one.

This graph makes three things precise:

1. **Bottom-up order = reverse topological sort** of the subproblem graph (a topological sort of its transpose): no subproblem is considered until everything it depends on is solved.
2. **Top-down memoization = depth-first search** of the subproblem graph.
3. **Running time** = `Θ(V + E)` in the common case, where the time to solve a subproblem is proportional to its out-degree.

Informally: **running time ≈ (number of subproblems) × (choices per subproblem).**

PROBLEM	SUBPROBLEMS	CHOICES EACH	TIME
Rod cutting	`Θ(n)`	`≤ n`	`Θ(n²)`
Matrix chain	`Θ(n²)`	`≤ n−1`	`Θ(n³)`
LCS	`Θ(mn)`	`O(1)`	`Θ(mn)`
Optimal BST	`Θ(n²)`	`≤ n`	`Θ(n³)`

Skiena's footnote says the same thing in one line: *"Suppose we create a graph with a vertex for every matrix cell, and a directed edge `(x, y)` when the value of cell `x` is needed to compute the value of cell `y`. Any topological sort on the resulting DAG (why must it be a DAG?) defines an acceptable evaluation order."* **The answer to his parenthetical: if it had a cycle, the recurrence would be circular — a subproblem would depend on itself, and no evaluation order would exist.** That is exactly the failure mode in §"When DP fails" below.

> *Outside / Engineering Context — the Galil–Park classification*
>
> *Galil and Park classify DP algorithms as* `tD/eD`*: table size* $O(n^t)$*, each entry depending on* $O(n^e)$ *others. LCS is* `2D/0D` *(quadratic table, constant work per cell); matrix chain is* `2D/1D` *(quadratic table, linear work per cell). This is a useful shorthand when comparing recurrences and when hunting for speedups — most classical DP optimizations (divide-and-conquer optimization, Knuth's optimization, convex-hull trick, SMAWK) are techniques for turning a* `?D/1D` *into a* `?D/0D`*.*

Part 2 — The Two Hallmarks (CLRS 14.3)

For DP to apply, a problem must have **both**:

Hallmark 1: Optimal substructure

An optimal solution to the problem contains within it optimal solutions to subproblems.

The four-step discovery pattern (memorize this — it is how you *find* the substructure, not just verify it):

1. Show that a solution consists of **making a choice** (which cut, which split point, which root). Making the choice leaves one or more subproblems.
2. **Suppose you are given** the choice that leads to an optimal solution. Don't worry yet how to find it.
3. Given the choice, determine **which subproblems ensue** and how best to characterize the resulting space of subproblems.
4. Show the subproblem solutions used inside an optimal solution **must themselves be optimal**, by **cut-and-paste**: assume one isn't, cut it out, paste in a better one, and derive a better overall solution — contradicting optimality.

Rule of thumb for the subproblem space: *keep it as simple as possible and expand only as necessary.* Rod cutting needed only "a rod of length `i`" — one index. Matrix chain seems like it might work with $A_1A_2\cdots A_j$ (one index), but it doesn't: splitting at `k` leaves $A_{k+1}\cdots A_j$, which is **not** of that form. You must let the subproblems **vary at both ends**. Getting this wrong is the single most common way to derive a DP that can't be closed.

Optimal substructure varies along two axes: how many subproblems an optimal solution uses, and how many choices you have. Rod cutting: **1 subproblem, `n` choices.** Matrix chain: **2 subproblems, `j - i` choices.**

Hallmark 2: Overlapping subproblems

The space of subproblems must be "small" — typically **polynomial** in the input size — *so that a naive recursion revisits the same subproblems over and over.*

Skiena states this as step 2 of his procedure: "**Show that the number of different parameter values taken on by your recurrence is bounded by a (hopefully small) polynomial.**" For edit distance: there are only `|P|·|T|` distinct `(i, j)` pairs, so at most that many distinct recursive calls.

Contrast: divide-and-conquer generates **brand-new** subproblems at each step. That's why memoizing merge sort buys nothing.

Quantitatively, on the two running examples:

- `CUT-ROD(p, n)` makes `T(n) = 1 + Σ_{j=0}^{n-1} T(j) = 2^n` calls. Exponential.
- `RECURSIVE-MATRIX-CHAIN(p, 1, n)` satisfies `T(n) ≥ 2 Σ_{i=1}^{n-1} T(i) + n`, which the substitution method resolves to `T(n) ≥ 2^{n-1}`, i.e. `Ω(2^n)`. **Memoization turns that `Ω(2^n)` into `O(n^3)`.**

The footnote that resolves the apparent contradiction

> *"It may seem strange that dynamic programming relies on subproblems being both independent and overlapping. Although these requirements may sound contradictory, they describe two different notions, rather than two points on the same axis. Two subproblems of the same problem are **independent** if they do not share resources. Two subproblems are **overlapping** if they are really the same subproblem that occurs as a subproblem of different problems."*

Say that out loud once and the confusion never comes back.

The subtlety: shortest path vs. longest simple path

Two problems on a directed graph `G` with `u, v ∈ V`:

Unweighted shortest path — *has* optimal substructure. Decompose `u ⇝ v` at any intermediate `w` into `u ⇝ w ⇝ v`. If the whole path is shortest, each piece must be shortest, by cut-and-paste.

Unweighted longest simple path — does not. CLRS's counterexample (Figure 14.6, four vertices `q, r, s, t`): `q → r → t` is a longest simple path from `q` to `t`. But `q → r` is **not** a longest simple path from `q` to `r` — `q → s → t → r` is longer. And `r → t` is not longest either — `r → q → s → t` is.

Worse: you can't even assemble a legal solution. Splicing `q → s → t → r` with `r → q → s → t` gives `q → s → t → r → q → s → t`, which is **not simple**.

Why the difference? *Independence.* For longest simple paths, the two subproblems **share resources** (vertices): using `s` and `t` in the first subproblem makes them unavailable to the second. For shortest paths the subproblems provably don't share: if some $x \neq w$ appeared in both p_1 and p_2, you could excise the $x \rightsquigarrow w \rightsquigarrow x$ loop and get a strictly shorter path — contradiction.

(No efficient DP for longest simple path has ever been found. It is NP-complete — M19.)

Skiena's version: the principle of optimality

> *"Dynamic programming can be applied to any problem that obeys the principle of optimality. Roughly stated, this means that partial solutions can be optimally extended given the state after the partial solution, instead of the specifics of the partial solution itself."*

For edit distance, deciding whether to substitute/insert/delete needs only the **cost** reached at `(i, j)`, not *which* sequence of operations got there. **Future decisions are made based on the consequences of previous decisions, not the actual decisions themselves.** Problems violating this are ones where the specifics of the operations matter — e.g. an edit distance forbidding certain *orders* of operations.

When DP fails — two distinct failure modes

Skiena's longest-simple-path example is the perfect teaching case, and it fails in **two independent ways**:

Failure 1 — the recurrence is wrong. Define `LP[i,j] = max_{(x,j)∈E} LP[i,x] + c(x,j)`. This *"does nothing to enforce simplicity"* — nothing stops vertex `j` from already appearing on the path from `i` to `x`, creating a cycle. **DP algorithms are only as correct as the recurrences they are based on.**

Failure 2 — there is no evaluation order. *"Because there is no left-to-right or smaller-to-bigger ordering of the vertices on the graph, it is not clear what the smaller subproblems are. Without such an ordering, we get stuck in an infinite loop as soon as we try to do anything."* The subproblem graph has a **cycle**, so no topological sort exists.

The repair, and what it costs. Put the visited set into the state: `LP'[i, j, S]` = longest simple path from `i` to `j` using intermediate set `S`. Now the recurrence is correct *and* orderable (states grow by one vertex at a time), but there are 2^n subsets. That's **Held-Karp** — exponential, but 2^n beats $n!$ by enough that *"this method can be used to solve TSPs for up to thirty vertices or more, where $n = 20$ would be impossible using the $O(n!)$ algorithm."*

*(Note the intermediate step Skiena walks through: storing the whole ordered path P_{ij} also gives a correct recurrence, but there are $(n-3)!$ of them — that's just backtracking wearing a DP costume. Collapsing the ordered path to an unordered **set** is the entire gain.)*

> ***Take-Home Lesson (Skiena):*** *"Without an inherent left-to-right ordering on the objects, dynamic programming is usually doomed to require exponential space and time."*
>
> ***And its positive form:*** *"For optimization problems on left-to-right objects, such as characters in a string, elements of a permutation, points around a polygon, or leaves in a search tree, dynamic programming likely leads to an efficient algorithm to find the optimal solution."*

The two design procedures, side by side

CLRS'S FOUR STEPS	SKIENA'S THREE STEPS
1. Characterize the structure of an optimal solution	1. Formulate the answer as a **recurrence relation** or recursive algorithm
2. Recursively define the value of an optimal solution	
3. Compute the value, typically bottom-up	2. Show the number of **distinct parameter values** is a small polynomial
4. Construct an optimal solution from computed information	3. Specify an **evaluation order** so partial results are ready when needed

They are the same procedure. CLRS front-loads the correctness argument; Skiena front-loads the mechanics. **Use CLRS's step 1 to convince yourself the recurrence is right, and Skiena's steps 2–3 to convince yourself it's fast.**

Part 3 — The Canonical Problems

3.1 Rod cutting (CLRS 14.1)

Problem. Given a rod of length `n` and prices `p₁..pₙ`, cut it to maximize total revenue. Cuts are free.

Why brute force fails: there are `n-1` possible cut positions, each independently taken or not, so **`2^{n-1}` decompositions**.

Recurrence. Two equivalent forms. The natural one:

```
rₙ = max{ pₙ, r₁ + r_{n-1}, r₂ + r_{n-2}, …, r_{n-1} + r₁ }
(14.1)
```

and the simpler one — **view a decomposition as a first piece of length `i` plus an undivided-decision remainder**:

```
rₙ = max{ pᵢ + r_{n-i} : 1 ≤ i ≤ n }
(14.2)
```

(14.2) is strictly better to work with: **one subproblem instead of two.** That reframing — "commit to the first piece, recurse only on the rest" — is a reusable move.

Complexity: `Θ(n)` subproblems × `≤ n` choices = **`Θ(n²)`**.

Reconstruction: store `s[j]` = the optimal *first* piece length for a rod of length `j`; then repeatedly print `s[n]` and set `n ← n − s[n]`.

```cpp
#include <climits>
#include <vector>

struct RodResult {
    long long revenue;
    std::vector<int> pieces;
};

RodResult rodCutting(const std::vector<long long>& price, int n) {
    std::vector<long long> r(n + 1, 0);
```

```
    std::vector<int> s(n + 1, 0);                  // best first-
piece length
    for (int j = 1; j <= n; ++j) {
        long long best = LLONG_MIN;
        for (int i = 1; i <= j && i < (int)price.size(); ++i) {
            if (best < price[i] + r[j - i]) {
                best = price[i] + r[j - i];
                s[j] = i;
            }
        }
        r[j] = best;
    }
    RodResult out{r[n], {}};
    for (int len = n; len > 0; len -= s[len])
out.pieces.push_back(s[len]);
    return out;
}
```

Verified: reproduces CLRS's table $r_1..r_{10} = 1, 5, 8, 10, 13, 17, 18, 22, 25, 30$, with reconstructed pieces summing to n and to the stated revenue; 60 random price tables match brute force.

> ***Exercise 14.1-2 is worth doing in your head:*** *the greedy "cut off the piece with the highest density p_i/i first" strategy is* ***wrong****. With CLRS's prices, length 4 has densities $1, 2.5, 2.67, 2.25$; greedy takes the length-3 piece (density 2.67) leaving a 1, for $8 + 1 = 9$ — but $2 + 2 = 10$ is optimal.* ***Optimal substructure does not imply greedy works*** *(M12).*
>
> ***Exercise 14.3-5 is the flip side:*** *add a limit l_i on how many pieces of length i you may produce, and* ***optimal substructure is destroyed*** *— the subproblem is no longer "a rod of length j" but "a rod of length j with the remaining quota", because the two sides now share a resource. Same lesson as longest simple path.*

3.2 Matrix-chain multiplication (CLRS 14.2)

Problem. Given $\langle A_1, \ldots, A_n \rangle$ with A_i of dimension $p_{i-1} \times p_i$, parenthesize the product to minimize scalar multiplications. Multiplying a $p\times q$ by a $q\times r$ costs pqr.

Why this matters at all: $\langle 10\times100,\ 100\times5,\ 5\times50 \rangle$ costs **7 500** as $((A_1A_2)A_3)$ and **75 000** as $(A_1(A_2A_3))$ — a **10× difference from parentheses alone.**

Why brute force fails: `P(n) = Σ_{k=1}^{n-1} P(k)P(n-k)` — the Catalan recurrence, `Ω(4^n/n^{3/2})`, and certainly `Ω(2^n)`.

Recurrence. `m[i,j]` = min cost to compute `A_i…A_j`:

```
              ⎧ 0
          if i = j
m[i,j]  =     ⎨
          (14.7)
              ⎩ min{ m[i,k] + m[k+1,j] + p_{i-1}·p_k·p_j  :  i ≤ k < j }     if i < j
```

The evaluation order is the interesting part. `m[i,j]` needs `m[i,k]` and `m[k+1,j]`, both of which cover **shorter chains**. So **iterate over chain length**, not over `i` or `j`. This "interval DP" loop shape — `for len; for i; j = i+len-1; for k in [i, j)` — recurs in optimal BST, CYK, polygon triangulation, burst balloons, and every other problem whose subproblems are contiguous ranges. **Memorize the loop skeleton.**

Complexity: `Θ(n²)` subproblems × `≤ n-1` choices = **`Θ(n³)`** (tight, by Exercise 14.2-5, which shows the total number of table references is exactly `(n³ - n)/3`).

```cpp
#include <climits>
#include <string>
#include <vector>

struct ChainResult {
    long long cost;
    std::string parens;
};

static void printParens(const std::vector<std::vector<int>>& s, int
i, int j, std::string& out) {
    if (i == j) { out += "A" + std::to_string(i + 1); return; }
    out += '(';
    printParens(s, i, s[i][j], out);
    printParens(s, s[i][j] + 1, j, out);
    out += ')';
}

// p has n+1 entries: matrix i (0-based) is p[i] x p[i+1]
ChainResult matrixChainOrder(const std::vector<long long>& p) {
```

```
    const int n = (int)p.size() - 1;
    std::vector<std::vector<long long>> m(n, std::vector<long long>
(n, 0));
    std::vector<std::vector<int>> s(n, std::vector<int>(n, 0));
    for (int len = 2; len <= n; ++len) {                // len =
chain length
        for (int i = 0; i + len - 1 < n; ++i) {
            const int j = i + len - 1;
            m[i][j] = LLONG_MAX;
            for (int k = i; k < j; ++k) {               // split
between A_k and A_{k+1}
                const long long q = m[i][k] + m[k + 1][j] + p[i] *
p[k + 1] * p[j + 1];
                if (q < m[i][j]) { m[i][j] = q; s[i][j] = k; }
            }
        }
    }
    std::string out;
    printParens(s, 0, n - 1, out);
    return {m[0][n - 1], out};
}
```

Verified: on CLRS's own instance `p = ⟨30, 35, 15, 5, 10, 20, 25⟩` this returns cost **15 125** with parenthesization `((A1(A2A3))((A4A5)A6))` — exactly the book's answer; 60 random chains match brute force.

> ***Exercise 14.3-4 is the greedy trap again:*** *Professor Capulet proposes choosing `k` to minimize `p_{i-1}p_k p_j` before solving the subproblems. It fails.* ***The cost attributable to the choice itself is not the whole cost.***

3.3 Longest common subsequence (CLRS 14.4)

Problem. Given `X = ⟨x₁..x_m⟩` and `Y = ⟨y₁..y_n⟩`, find a longest sequence that is a subsequence of both. (Subsequence: order preserved, contiguity not required.)

Theorem 14.1 (optimal substructure of an LCS). Let `Z = ⟨z₁..z_k⟩` be *any* LCS of `X` and `Y`.

1. If `x_m = y_n`, then `z_k = x_m = y_n` **and** `Z_{k-1}` is an LCS of `X_{m-1}` and `Y_{n-1}`.

2. If `x_m ≠ y_n` and `z_k ≠ x_m`, then `Z` is an LCS of `X_{m−1}` and `Y`.
3. If `x_m ≠ y_n` and `z_k ≠ y_n`, then `Z` is an LCS of `X` and `Y_{n−1}`.

Proof skeleton. (1) If `z_k ≠ x_m` we could append `x_m = y_n` to `Z` and get a longer common subsequence — contradiction. And if some `W` longer than `Z_{k−1}` were a common subsequence of `X_{m−1}, Y_{n−1}`, appending `x_m` to `W` beats `Z`. (2) If `z_k ≠ x_m` then `Z` is common to `X_{m−1}` and `Y`; anything longer would also be common to `X` and `Y`. (3) Symmetric. ∎

Recurrence.

```
            ⎧ 0                                  if i = 0 or j = 0
c[i,j]  =   ⎨ c[i−1,j−1] + 1                     if i,j > 0 and xᵢ =
yⱼ             (14.9)
            ⎩ max{ c[i,j−1], c[i−1,j] }          if i,j > 0 and xᵢ ≠
yⱼ
```

Note what's new here: *"a condition in the problem restricts which subproblems to consider."* When `xᵢ = yⱼ` you consider **only** the diagonal subproblem — you don't take a max over three. (Edit distance has the same character.) Rod cutting and matrix chain never ruled out subproblems this way.

Complexity: `Θ(mn)` subproblems, `O(1)` work each ⟹ **`Θ(mn)` time and space.** (A `2D/0D` algorithm in Galil–Park terms.)

Two improvements, and the tension between them:

- **Drop the `b` (arrow) table.** Each `c[i,j]` depends on only three neighbors, so you can re-derive the choice in `O(1)` from `c` alone. Saves `Θ(mn)` space but the `c` table still costs `Θ(mn)`, so no asymptotic win — just simpler code.
- **Keep only two rows** of `c`. Now the space really is `Θ(min(m,n))`. **But this breaks reconstruction**: the smaller table doesn't retain enough to retrace.

> ### *Outside / Engineering Context — Hirschberg's algorithm*
>
> *You can have both: `Θ(mn)` time, `Θ(min(m,n))` space, **and** the actual alignment, via a divide-and-conquer trick. Compute the last row of the DP for the first half of `X` forward and for the second half of `X` backward; their sum is minimized at the column where the optimal alignment crosses the midline. Recurse on the two halves. Total work is `mn + mn/2 + mn/4 + … = 2mn`. Skiena flags this as "a clever divide-and-conquer algorithm that computes the actual alignment in the same `O(nm)` time but*

> *only `O(m)` space", and notes why it matters: "Since memory on any computer is limited, using `O(nm)` space proves more of a bottleneck than `O(nm)` time." This is what `diff` and bioinformatics aligners actually use.*

```
#include <algorithm>
#include <string>
#include <vector>

std::vector<std::vector<int>> lcsTable(const std::string& x, const
std::string& y) {
    const int m = (int)x.size(), n = (int)y.size();
    std::vector<std::vector<int>> c(m + 1, std::vector<int>(n + 1,
0));
    for (int i = 1; i <= m; ++i)
        for (int j = 1; j <= n; ++j)
            c[i][j] = (x[i - 1] == y[j - 1]) ? c[i - 1][j - 1] + 1
                                             : std::max(c[i - 1]
[j], c[i][j - 1]);
    return c;
}

// Reconstruct straight from the cost table (no separate b table
needed).
std::string lcsString(const std::string& x, const std::string& y) {
    const auto c = lcsTable(x, y);
    int i = (int)x.size(), j = (int)y.size();
    std::string out;
    while (i > 0 && j > 0) {
        if (x[i - 1] == y[j - 1]) { out += x[i - 1]; --i; --j; }
        else if (c[i - 1][j] >= c[i][j - 1]) --i;
        else --j;
    }
    std::reverse(out.begin(), out.end());
    return out;
}

// Length only, in O(min(m,n)) space: two rolling rows.
int lcsLengthSmallSpace(const std::string& x, const std::string& y)
{
    const std::string& a = x.size() >= y.size() ? x : y;   //
iterate over the longer
```

```
    const std::string& b = x.size() >= y.size() ? y : x;   // rows
indexed by the shorter
    std::vector<int> prev(b.size() + 1, 0), cur(b.size() + 1, 0);
    for (std::size_t i = 1; i <= a.size(); ++i) {
        for (std::size_t j = 1; j <= b.size(); ++j)
            cur[j] = (a[i - 1] == b[j - 1]) ? prev[j - 1] + 1
                                            : std::max(prev[j],
cur[j - 1]);
        prev.swap(cur);
    }
    return prev[b.size()];
}
```

Verified: `LCS("ABCBDAB", "BDCABA") = 4` (CLRS's instance); across 200 random pairs the three routines agree with each other and with exhaustive enumeration of all `2^m` subsequences, and every reconstructed string was checked to actually be a subsequence of both inputs.

3.4 Edit distance — and why it is the most valuable recurrence in the chapter

Problem. Convert pattern `P` into text `T` using **substitution**, **insertion**, and **deletion**. With unit costs this is the **edit distance** (Levenshtein distance).

The derivation, which is the model for how to find any DP recurrence. *"To solve it, let's think about the problem in reverse. What information would we need to select the final operation correctly? What can happen to the last character in the matching for each string?"*

There are exactly **three** possibilities for the last character, and no others:

OPERATION	RECURRENCE TERM	MEANING
match / substitute	`D[i−1, j−1] + match(Pᵢ, Tⱼ)`	consume one from each
insertion	`D[i, j−1] + indel(Tⱼ)`	an extra character in the text
deletion	`D[i−1, j] + indel(Pᵢ)`	an extra character in the pattern

```
D[i, j] = min{ D[i−1,j−1] + match(Pᵢ,Tⱼ),  D[i,j−1] + indel(Tⱼ),
D[i−1,j] + indel(Pᵢ) }
```

The naive recursion is worse than exponential — *"it grows at a rate of at least* 3^n *— indeed, even faster since most of the calls reduce only one of the two indices, not both."* Skiena reports it taking **several seconds to compare two 11-character strings**. But there can only be `|P|·|T|` distinct `(i, j)` pairs, so the table has `θ(mn)` cells.

Evaluation order: `(i,j)` needs `(i−1,j−1)`, `(i,j−1)`, `(i−1,j)`. **Any order with that property works**, including plain row-major.

The four stub functions — the real reason to learn this

Skiena factors the algorithm into four replaceable pieces. **Swapping them turns one implementation into half a dozen algorithms.**

STUB	STANDARD EDIT DISTANCE	PURPOSE
`row_init(i)` / `column_init(i)`	`m[0][i] = i`, `m[i][0] = i`	boundary: matching a length-`i` string against the empty string costs `i` indels
`match(c,d)` / `indel(c)`	`0` if `c == d` else `1`; `indel` = 1	the cost model
`goal_cell()`	`(|P|, |T|)`	where the answer lives
`match_out` / `insert_out` / `delete_out`	print `M`/`S`/`I`/`D`	what to do during traceback

Variant 1 — approximate substring matching. Find where a short pattern best occurs *inside* a long text. Plain edit distance is useless here: *"the vast majority of any edit cost will consist of deleting all that is not 'Skiena' from the body of the text."* The fix is two stub changes:

- `row_init(i): m[0][i].cost = 0` — **starting a match anywhere in the text is free**;
- `goal_cell()`: scan the **last row** for its minimum, rather than taking `(m, n)`.

Variant 2 — LCS. Forbid substitution by making it expensive: `match(c,d) = MAXLEN` when `c ≠ d`. Then the only way to reconcile non-matching characters is insert+delete, so the minimum-cost alignment maximizes the number of true matches. *(It suffices for the substitution penalty to exceed one insertion plus one deletion.)* The identity to check: **`indels = |a| + |b| − 2·LCS(a,b)`**.

Variant 3 — longest increasing subsequence. LIS of `s` is the **LCS of `s` with `s` sorted ascending**. Any common subsequence must respect both the order in `s` and increasing value order. (Reverse the sort for longest decreasing.)

> *"As you can see, our edit distance routine can be made to do many amazing things easily. The trick is observing that your problem is just a special case of approximate string matching."*
>
> *And the caution: "we must confess it is difficult to get the boundary conditions and index manipulations correct. Although dynamic programming algorithms are easy to design once you understand the technique, getting the details right requires clear thinking and thorough testing."*

```
#include <algorithm>
#include <functional>
#include <string>
#include <vector>

enum EditOp { OP_MATCH = 0, OP_INSERT = 1, OP_DELETE = 2, OP_NONE =
-1 };

struct EditResult {
    int cost;
    std::string trace;          // M / S / I / D, in forward order
};

// rowInitZero = true makes the cost of starting a match anywhere
in t free,
// which turns edit distance into approximate substring matching.
EditResult editDistance(const std::string& s, const std::string& t,
                        bool rowInitZero = false, int substCost =
1) {
    const int m = (int)s.size(), n = (int)t.size();
    std::vector<std::vector<int>> cost(m + 1, std::vector<int>(n +
1, 0));
    std::vector<std::vector<int>> parent(m + 1, std::vector<int>(n
+ 1, OP_NONE));

    for (int j = 0; j <= n; ++j) {                      // row 0: t
consumed by insertions
        cost[0][j] = rowInitZero ? 0 : j;
```

```
        parent[0][j] = (j > 0 && !rowInitZero) ? OP_INSERT :
OP_NONE;
    }
    for (int i = 1; i <= m; ++i) {                      // column
0: s consumed by deletions
        cost[i][0] = i;
        parent[i][0] = OP_DELETE;
    }
    for (int i = 1; i <= m; ++i) {
        for (int j = 1; j <= n; ++j) {
            const int match = (s[i - 1] == t[j - 1]) ? 0 :
substCost;
            int opt[3];
            opt[OP_MATCH]  = cost[i - 1][j - 1] + match;
            opt[OP_INSERT] = cost[i][j - 1] + 1;
            opt[OP_DELETE] = cost[i - 1][j] + 1;
            cost[i][j] = opt[OP_MATCH];
            parent[i][j] = OP_MATCH;
            for (int k = OP_INSERT; k <= OP_DELETE; ++k)
                if (opt[k] < cost[i][j]) { cost[i][j] = opt[k];
parent[i][j] = k; }
        }
    }
    // goal cell: the end of both strings, or the cheapest column
of the last row
    int gi = m, gj = n;
    if (rowInitZero)
        for (int j = 0; j <= n; ++j) if (cost[m][j] < cost[gi][gj])
gj = j;

    std::string trace;
    std::function<void(int, int)> walk = [&](int i, int j) {
        if (parent[i][j] == OP_NONE) return;
        if (parent[i][j] == OP_MATCH) {
            walk(i - 1, j - 1);
            trace += (s[i - 1] == t[j - 1]) ? 'M' : 'S';
        } else if (parent[i][j] == OP_INSERT) {
            walk(i, j - 1);
            trace += 'I';
        } else {
            walk(i - 1, j);
            trace += 'D';
```

```
        }
    };
    walk(gi, gj);
    return {cost[gi][gj], trace};
}
```

Note the reconstruction idiom: recursing *before* emitting reverses the backward walk for free — *"clever use of recursion can do the reversing for us."* Same trick as `PRINT-LCS` and `reconstruct_partition`.

Verified: `editDistance("thou shalt", "you should")` returns cost **5** with trace **`DSMMMMMISMS`** — character-for-character the sequence in Skiena's Figure 10.6 (*delete the first "t"; replace "h" with "y"; match five; insert an "o"; replace "a" with "u"; replace "t" with "d"*). 200 random pairs match a brute-force recursion; the `substCost = 1000` trick reproduces `|a| + |b| − 2·LCS` exactly on 100 random pairs; and approximate substring search finds the misspelling `"Skeina"` inside `"xxxxxSkeinaxxxxxxxxxx"` at cost 2.

> ***For most problems, including edit distance, you can reconstruct without a parent table*** *by working backward from the three ancestor costs and the characters. "But it is cleaner and easier to explicitly store the moves." Do that under interview pressure.*

3.5 Longest increasing subsequence (Skiena 10.3)

Skiena reworks LIS from scratch even though it's a special case of edit distance, because *"it is instructive to work it out from scratch. Indeed, dynamic programming algorithms are often easier to reinvent than look up."*

Finding the recurrence — watch the reasoning, it's the transferable part.

Attempt 1: let `L` = the length of the LIS in $s_1..s_{n-1}$. Not enough. *"Suppose I told you that the longest increasing sequence in* $(s_1,\ldots,s_{n-1})$ *was of length 5 and that* $s_n = 8$*. Will the length of the LIS of* s *be 5 or 6? It depends on whether the length-5 sequence ended with a value* < 8*."*

Attempt 2: **"We need to know the length of the longest sequence that s_n will extend. To be certain we know this, we really need the length of the longest sequence ending at every possible value s_i."**

That is the whole insight, and it is the archetype of DP state design: **the state must record whatever the next decision depends on.**

```
Lᵢ = 1 + max{ Lⱼ : 0 ≤ j < i and sⱼ < sᵢ },     L₀ = 0
answer = max_{1≤i≤n} Lᵢ                          (the winning sequence
must end somewhere)
```

Skiena's worked table:

INDEX i	1	2	3	4	5	6	7	8	9
s_i	2	4	3	5	1	7	6	9	8
L_i	1	2	2	3	1	4	4	5	5
p_i	–	1	1	2	–	4	4	6	6

Complexity: `n` values, each compared against `≤ n` predecessors ⟹ **$O(n^2)$**.
Reconstruction: store the predecessor index p_i, then follow the chain back from the best endpoint.

The `O(n lg n)` version (CLRS Exercise 14.4-6). Maintain `tails[k]` = the *smallest possible tail value* of any increasing subsequence of length `k+1`. `tails` is automatically sorted, so binary-search for the first element `≥ aᵢ` and overwrite it (or append). The array's **length** is the answer; the array's **contents are not the LIS**, so reconstruction needs a separate predecessor chain — which is the part everyone forgets.

```cpp
#include <algorithm>
#include <vector>

// O(n^2): L[i] = length of the LIS ending at i, p[i] = predecessor
index.
std::vector<int> lisQuadratic(const std::vector<int>& a) {
    const int n = (int)a.size();
    if (n == 0) return {};
    std::vector<int> L(n, 1), p(n, -1);
    for (int i = 0; i < n; ++i)
        for (int j = 0; j < i; ++j)
            if (a[j] < a[i] && L[j] + 1 > L[i]) { L[i] = L[j] + 1;
p[i] = j; }
    const int best = (int)(std::max_element(L.begin(), L.end()) -
L.begin());
    std::vector<int> out;
    for (int i = best; i >= 0; i = p[i]) out.push_back(a[i]);
    std::reverse(out.begin(), out.end());
    return out;
```

```
}

// O(n lg n): tails[k] = smallest possible tail of an increasing
subsequence of length k+1.
std::vector<int> lisNLogN(const std::vector<int>& a) {
    const int n = (int)a.size();
    if (n == 0) return {};
    std::vector<int> tails;                    // values, strictly
increasing
    std::vector<int> tailIdx;                  // index in a of
each tail
    std::vector<int> prev(n, -1);
    for (int i = 0; i < n; ++i) {
        const int k = (int)(std::lower_bound(tails.begin(),
tails.end(), a[i]) - tails.begin());
        if (k > 0) prev[i] = tailIdx[k - 1];
        if (k == (int)tails.size()) { tails.push_back(a[i]);
tailIdx.push_back(i); }
        else                        { tails[k] = a[i];
tailIdx[k] = i; }
    }
    std::vector<int> out;
    for (int i = tailIdx.back(); i >= 0; i = prev[i])
out.push_back(a[i]);
    std::reverse(out.begin(), out.end());
    return out;
}
```

Verified: on Skiena's `(2,4,3,5,1,7,6,9,8)` both give length 5; across 300 random inputs the two agree with each other and with exhaustive subset enumeration, and every reconstruction was checked to be strictly increasing **and** an actual subsequence of the input.

> *Use `lower_bound` for **strictly** increasing, `upper_bound` for **non-decreasing**. Getting that one call wrong is the classic LIS bug.*

3.6 Subset sum and 0-1 knapsack (Skiena 10.5)

Problem. Does some subset of $S = \{s_1, \ldots, s_n\}$ sum to exactly k?

The recurrence follows from a single binary question: *is s_n in the subset or not?*

```
T[n, k] = T[n−1, k]  ∨  T[n−1, k − sₙ]
```

If it is, the first `n−1` must make `k − sₙ`. If not, the first `n−1` must make `k` alone. **No overlap between the cases, and all possibilities covered.**

Complexity: `θ(nk)` cells, `O(1)` each. **Reconstruction:** a `parent` table recording `j − s[i−1]` when item `i` was used, `NIL` otherwise; walk up rows until you find a non-`NIL`.

The pseudo-polynomial trap — say this correctly or not at all.

> *"The alert reader might wonder how we can have an `O(nk)` algorithm for subset sum when subset sum is an NP-complete problem? Isn't this polynomial in `n` and `k`? Did we just prove that `P = NP`? Unfortunately, no."*

The target `k` is specified in `O(log k)` bits, so the algorithm runs in time **exponential in the size of the input**, which is `O(n log k)`. Skiena's test for whether you've internalized it: *"consider what happens to the algorithm when we take a specific problem instance and multiply each integer by 1,000,000. Such a transform would not have affected the running time of sorting or minimum spanning tree... But it would slow down our dynamic programming algorithm by a factor of 1,000,000."* **The range of the numbers matters. That is the signature of a pseudo-polynomial algorithm.** (Same reason trial-dividing `N` by all `√N` candidates isn't polynomial factoring.)

```cpp
#include <algorithm>
#include <vector>

struct SubsetSumResult {
    bool feasible;
    std::vector<int> chosen;                    // the actual
values used
};

SubsetSumResult subsetSum(const std::vector<int>& s, int k) {
    const int n = (int)s.size();
    std::vector<std::vector<char>> can(n + 1, std::vector<char>(k +
1, 0));
    can[0][0] = 1;
    for (int i = 1; i <= n; ++i)
        for (int j = 0; j <= k; ++j)
            can[i][j] = can[i - 1][j] || (j >= s[i - 1] && can[i -
1][j - s[i - 1]]);
```

```
    SubsetSumResult out{can[n][k] != 0, {}};
    if (!out.feasible) return out;
    int j = k;
    for (int i = n; i > 0; --i)
        if (!can[i - 1][j]) { out.chosen.push_back(s[i - 1]); j -=
s[i - 1]; }
    std::reverse(out.chosen.begin(), out.chosen.end());
    return out;
}

// 0-1 knapsack, rolling 1-D array (iterate capacity downward so
each item is used once)
long long knapsack01(const std::vector<int>& weight, const
std::vector<long long>& value, int cap) {
    std::vector<long long> best(cap + 1, 0);
    for (std::size_t i = 0; i < weight.size(); ++i)
        for (int c = cap; c >= weight[i]; --c)
            best[c] = std::max(best[c], best[c - weight[i]] +
value[i]);
    return best[cap];
}
```

The downward inner loop in `knapsack01` is load-bearing. Iterating `c` upward would let the same item be picked more than once — which is exactly the **unbounded** knapsack. One loop direction distinguishes two different problems; know which you want.

Verified: `S = {1,2,4,8}`, `k = 11` returns `{1,2,8}`, and (as Skiena notes, since these are the powers of two and every integer has a binary representation) the whole bottom row of the table is `true`. 200 random subset-sum instances and 200 random knapsacks match exhaustive subset enumeration.

> ## *Outside / Engineering Context*
>
> - ***Bitset speedup.*** *For pure feasibility,* `std::bitset<K> reach; reach[0] = 1; for (int x : s) reach |= reach << x;` *runs in* `O(nk/64)` *— a 64× constant-factor win, and the standard competitive-programming answer.*
> - ***Meet in the middle.*** *Split* `S` *into halves, enumerate all* `2^{n/2}` *subset sums of each, sort one and binary-search —* `O(2^{n/2} · n)`*. This is what you use when* `k` *is astronomically large but* `n ≤ 40`*.*
> - ***Knapsack by value.*** *When values are small but weights are huge, flip the table:*

> *`minWeight[v]` = least weight achieving value `v`. `O(n · Σvᵢ)`. This flip is the basis of the FPTAS for knapsack (M20).*

3.7 The ordered partition problem (Skiena 10.7)

Problem (Integer Partition without Rearrangement). Given $s_1..s_n$ and k, partition into $\le k$ **contiguous** ranges minimizing the **maximum range sum**, without reordering.

The motivating story: three workers scanning a shelf of books. Equal-count partitioning is wrong when the books differ in length — `100 200 300 | 400 500 600 | 700 800 900` gives one worker 600 pages and another 2400. The fair split is `100 200 300 400 500 | 600 700 | 800 900`, max 1700.

> *"A novice algorist might suggest a heuristic... perhaps by computing the average weight of a partition $\Sigma s_i/k$, and then trying to insert the dividers to come close to this average. However, such heuristic methods are doomed to fail on certain inputs because they do not systematically evaluate all possibilities."*

Recurrence. `M[n,k]` = min possible cost over all partitions of $s_1..s_n$ into k ranges. **Ask where the last divider goes:**

```
M[n,k] = min_{i=1..n} max( M[i, k−1],  Σ_{j=i+1..n} sⱼ )
```

The cost is the larger of (a) the last partition's sum, and (b) the best you can do on the prefix with $k-1$ dividers.

Boundary conditions — Skiena is explicit that a recurrence isn't complete without them, and that they come from the *smallest reasonable value of each argument*:

```
M[1,k] = s₁    for all k > 0     (you can't make a first partition
smaller than s₁)
M[n,1] = Σᵢ sᵢ                    (no dividers at all)
```

Complexity. kn cells; naively each takes $O(n)$ splits × $O(n)$ to sum a range = $O(kn^3)$. **Prefix sums** $p_i = \Sigma_{j\le i} s_j$ make each range sum $p_j - p_{i-1}$ an $O(1)$ lookup, giving $\mathbf{O(kn^2)}$. *(Note the prefix sums also serve as the $k = 1$ initialization — a nice economy.)*

This problem is the parallel-processing load-balancing problem, and Skiena connects it to his own War Story in §5.8 ("Going Nowhere Fast", covered in M03) — *"the war story of Section 5.8 revolves around a botched solution to the very problem discussed here."*

```
#include <algorithm>
#include <climits>
#include <functional>
#include <vector>

struct PartitionResult {
    long long cost;                                  // largest range
sum
    std::vector<std::vector<int>> parts;
};

PartitionResult orderedPartition(const std::vector<int>& s, int k)
{
    const int n = (int)s.size();
    std::vector<long long> pre(n + 1, 0);          // prefix sums
make each cell O(1)
    for (int i = 1; i <= n; ++i) pre[i] = pre[i - 1] + s[i - 1];

    std::vector<std::vector<long long>> m(n + 1, std::vector<long
long>(k + 1, 0));
    std::vector<std::vector<int>> d(n + 1, std::vector<int>(k + 1,
0));
    for (int i = 1; i <= n; ++i) m[i][1] = pre[i];
    for (int j = 1; j <= k; ++j) m[1][j] = s[0];
    for (int i = 2; i <= n; ++i) {
        for (int j = 2; j <= k; ++j) {
            m[i][j] = LLONG_MAX;
            for (int x = 1; x <= i - 1; ++x) {
                const long long c = std::max(m[x][j - 1], pre[i] -
pre[x]);
                if (c < m[i][j]) { m[i][j] = c; d[i][j] = x; }
            }
        }
    }
    PartitionResult out{m[n][k], {}};
    std::function<void(int, int)> rebuild = [&](int i, int j) {
```

```
        if (j == 1) { out.parts.push_back(std::vector<int>
(s.begin(), s.begin() + i)); return; }
        rebuild(d[i][j], j - 1);
        out.parts.push_back(std::vector<int>(s.begin() + d[i][j],
s.begin() + i));
    };
    rebuild(n, k);
    return out;
}
```

Verified: `(1,2,…,9)` with `k = 3` returns max part **17** with the split `{1,2,3,4,5} {6,7} {8,9}` — Skiena's Figure 10.9 exactly; 200 random instances match brute-force enumeration of all divider placements, and the reconstructed parts were checked to concatenate back to the input.

> ### *Outside / Engineering Context*
>
> *This is LeetCode's "Split Array Largest Sum" / "Book Allocation" — and there is a slicker `O(n lg Σ)` solution: **binary search on the answer**, greedily checking feasibility of a candidate maximum. That's usually the expected interview answer. Know both: the DP generalizes to arbitrary cost functions (Skiena's War Story below relies on that), the binary search does not.*

3.8 Parsing context-free grammars — CYK (Skiena 10.8)

Problem. Given a context-free grammar `G` in **Chomsky normal form** (every rule is `X → Y Z` or `X → α`) and a string `s` of length `n`, is `s` derivable?

(Any CFG converts to Chomsky normal form mechanically by shortening long right-hand sides at the cost of extra nonterminals, so this is no loss of generality.)

The key observation: the rule applied at the **root** of the parse tree, say `X → Y Z`, splits `s` at some position `i` such that $s_1 \cdots s_i$ is generated by `Y` and $s_{i+1} \cdots s_n$ by `Z`. **This is exactly the matrix-chain structure** — an interval DP over contiguous subsequences.

```
M[i, j, X] = ∨_{(X→YZ) ∈ G}  ∨_{k=i}^{j−1}  ( M[i,k,Y] ∧ M[k+1,j,Z]
)
M[i, i, X] = true  iff  ∃ production X → α with Sᵢ = α
```

Complexity. `O(n²)` intervals × constant grammar size × `O(n)` split points = **`O(n³)`**. *(The grammar's size is constant because the grammar for C or Java is fixed regardless of program length.)*

Skiena's aside is a good one: *"Parsing seemed like a horribly complicated subject when I took a compilers course as a graduate student. But, more recently a friend easily explained it to me over lunch. The difference is that I understand dynamic programming much better now than when I was a student."*

Stop and Think: Parsimonious Parserization. *Given `G` and `s`, find the smallest number of character substitutions to make `s` parseable.* This "seemed extremely difficult when I first encountered it. But on reflection, it is just a very general version of edit distance." Replace `∨/∧` with `min/+`:

```
M'[i,j,X] = min_{(X→YZ)} min_{k=i}^{j−1} ( M'[i,k,Y] + M'[k+1,j,Z] )
M'[i,i,X] = 0 if some X → Sᵢ exists;  1 if some X → α exists with α ≠ Sᵢ;  ∞ if no X → α exists
```

Swapping the semiring — `(∨, ∧)` for `(min, +)` — turns a decision problem into an optimization problem with zero structural change. That move generalizes: `(max, +)` for longest, `(+, ×)` for counting, `(max, ×)` for most-probable (the Viterbi algorithm, CLRS Problem 14-7).

```cpp
#include <algorithm>
#include <array>
#include <string>
#include <utility>
#include <vector>

struct Grammar {
    int nonterminals = 0;
    std::vector<std::array<int, 3>> binary;      // X -> Y Z
    std::vector<std::pair<int, char>> unary;     // X -> a
};

// M[i][j][X] = can nonterminal X derive s[i..j]?
bool cykParse(const Grammar& g, const std::string& s, int start) {
    const int n = (int)s.size();
    if (n == 0) return false;
    std::vector<std::vector<std::vector<char>>> M(
```

```
        n, std::vector<std::vector<char>>(n, std::vector<char>
(g.nonterminals, 0)));
    for (int i = 0; i < n; ++i)
        for (const auto& u : g.unary)
            if (u.second == s[i]) M[i][i][u.first] = 1;
    for (int len = 2; len <= n; ++len)
        for (int i = 0; i + len - 1 < n; ++i) {
            const int j = i + len - 1;
            for (const auto& b : g.binary)
                for (int k = i; k < j && !M[i][j][b[0]]; ++k)
                    if (M[i][k][b[1]] && M[k + 1][j][b[2]]) M[i][j]
[b[0]] = 1;
        }
    return M[0][n - 1][start] != 0;
}

// Minimum single-character substitutions so that s is generated by
`start`.
int cykMinEdits(const Grammar& g, const std::string& s, int start)
{
    const int n = (int)s.size();
    const int INF = 1000000;
    if (n == 0) return INF;
    std::vector<std::vector<std::vector<int>>> M(
        n, std::vector<std::vector<int>>(n, std::vector<int>
(g.nonterminals, INF)));
    for (int i = 0; i < n; ++i)
        for (const auto& u : g.unary)
            M[i][i][u.first] = std::min(M[i][i][u.first], u.second
== s[i] ? 0 : 1);
    for (int len = 2; len <= n; ++len)
        for (int i = 0; i + len - 1 < n; ++i) {
            const int j = i + len - 1;
            for (const auto& b : g.binary)
                for (int k = i; k < j; ++k) {
                    const long long v = (long long)M[i][k][b[1]] +
M[k + 1][j][b[2]];
                    if (v < M[i][j][b[0]]) M[i][j][b[0]] = (int)v;
                }
        }
    return M[0][n - 1][start];
}
```

Verified on Skiena's own toy grammar (`sentence ::= noun-phrase verb-phrase`, `noun-phrase ::= article noun`, `verb-phrase ::= verb noun-phrase`, articles `the`/`a`, nouns `cat`/`milk`, verb `drank`, encoded one character per word): it accepts *"the cat drank the milk"*, rejects both truncations, and the min-substitution variant repairs a one-character corruption at cost 1 and a four-character corruption at cost 4. 300 random strings cross-checked against exhaustive derivation.

3.9 Optimal binary search trees (CLRS 14.5)

Problem. You know how often each key is searched. Build the BST minimizing **expected** search cost — not height.

Formally: keys $k_1 < \cdots < k_n$ with search probabilities p_i, plus $n+1$ **dummy keys** $d_0 .. d_n$ for unsuccessful searches with probabilities q_i, where $\Sigma p_i + \Sigma q_i = 1$. Cost of a search = number of nodes examined = depth + 1. So

```
E[search cost] = 1 + Σᵢ depth(kᵢ)·pᵢ + Σᵢ depth(dᵢ)·qᵢ
   (14.11)
```

Two facts worth internalizing (both from CLRS's Figure 14.9):

- **An optimal BST is not necessarily the one of smallest height.**
- **An optimal BST does not necessarily have the highest-probability key at the root.** In the book's example k_5 has the largest p (0.20) but the optimal root is k_2; the best tree with k_5 at the root costs 2.85 vs. the optimum 2.75.

Optimal substructure. Any subtree of a BST contains a **contiguous key range** $k_i .. k_j$ plus dummies $d_{i-1} .. d_j$. If T is optimal and T' is its subtree on $k_i .. k_j$, then T' must be optimal for that subproblem — the usual cut-and-paste.

The recurrence, and the one clever step. Choose a root k_r. When a subtree becomes a subtree of a node, **every node in it gets one deeper**, so its expected cost increases by the **sum of all probabilities in it**:

```
w(i,j) = Σ_{l=i..j} p_l + Σ_{l=i-1..j} q_l
     (14.12)
```

Then

```
e[i,j] = p_r + (e[i,r-1] + w(i,r-1)) + (e[r+1,j] + w(r+1,j))
```

and since `w(i,j) = w(i,r−1) + p_r + w(r+1,j)`, this collapses to the clean form

```
e[i,j] = e[i,r−1] + e[r+1,j] + w(i,j)
    (14.13)
```

giving

```
           ⎧ q_{i−1}                                                  if
j = i − 1
e[i,j]  =  ⎨
           ⎩ min{ e[i,r−1] + e[r+1,j] + w(i,j) : i ≤ r ≤ j }     if
i ≤ j
```

Watch the `w` collapse — that's the step that makes the recurrence `O(1)` per candidate root rather than `O(j−i)`. And `w` itself is computed incrementally: `w[i,j] = w[i,j−1] + pⱼ + qⱼ`, so all `Θ(n²)` values cost `Θ(1)` each. *(Exercise 14.5-3 asks what happens if you don't do this: `Θ(n⁴)`.)*

Index care: `e` is `[1..n+1, 0..n]` — the first index runs to `n+1` so `e[n+1,n]` (the subtree containing only d_n) exists, and the second starts at 0 so `e[1,0]` (only d_0) exists.

Complexity: `Θ(n³)`, exactly like matrix chain — same interval structure. **Exercise 14.5-4** cites **Knuth's** result that `root[i,j−1] ≤ root[i,j] ≤ root[i+1,j]`, which shrinks the inner loop and gives **`Θ(n²)`**. *(This is "Knuth's optimization", the standard `2D/1D → 2D/0D` speedup; it applies whenever the cost function is monotone and satisfies the quadrangle inequality.)*

```cpp
#include <limits>
#include <vector>

struct OptimalBSTResult {
    double cost;
    std::vector<std::vector<int>> root;          // root[i][j], 1-
based keys
};

OptimalBSTResult optimalBST(const std::vector<double>& p, const
std::vector<double>& q) {
    const int n = (int)p.size() - 1;             // p[1..n],
q[0..n]
```

```
    std::vector<std::vector<double>> e(n + 2, std::vector<double>(n
+ 1, 0));
    std::vector<std::vector<double>> w(n + 2, std::vector<double>(n
+ 1, 0));
    std::vector<std::vector<int>> root(n + 1, std::vector<int>(n +
1, 0));
    for (int i = 1; i <= n + 1; ++i) { e[i][i - 1] = q[i - 1]; w[i]
[i - 1] = q[i - 1]; }
    for (int len = 1; len <= n; ++len)
        for (int i = 1; i + len - 1 <= n; ++i) {
            const int j = i + len - 1;
            e[i][j] = std::numeric_limits<double>::infinity();
            w[i][j] = w[i][j - 1] + p[j] + q[j];
            for (int r = i; r <= j; ++r) {
                const double t = e[i][r - 1] + e[r + 1][j] + w[i]
[j];
                if (t < e[i][j]) { e[i][j] = t; root[i][j] = r; }
            }
        }
    return {e[1][n], root};
}
```

Verified: on CLRS's distribution the expected cost is **2.75** with k_2 **at the root** — matching Figure 14.9(b) precisely; 40 random distributions match exhaustive search over all Catalan-many tree shapes.

> ### *Outside / Engineering Context*
>
> ***Splay trees get within a constant factor of the optimal bound without knowing the frequencies at all*** *(self-adjusting; M08, M09). So the optimal-BST DP is the right tool only when the distribution is known and fixed — a static dictionary, a Huffman-adjacent encoding problem, or a compiler's keyword table. When access patterns drift, use a splay tree.*

3.10 Held-Karp: DP when there is no left-to-right order (Skiena 10.9.2)

Held-Karp is the canonical demonstration of both **how to repair a broken DP** and **what it costs**.

State: `dp[S][j]` = cheapest path starting at city 0, visiting exactly the set `S`, ending at `j`.
Transition: `dp[S ∪ {k}][k] = min over j ∈ S of dp[S][j] + d[j][k]`.
Answer: `min over j of dp[full][j] + d[j][0]`.

Complexity: $O(2^n \cdot n^2)$ time, $O(2^n \cdot n)$ space. Exponential — but 2^n is a vast improvement over $n!$ and makes $n \approx 20$–25 routine where $n = 20$ is hopeless by brute force ($20! \approx 2.4 \times 10^{18}$ vs $2^{20} \cdot 400 \approx 4 \times 10^8$).

```cpp
#include <algorithm>
#include <climits>
#include <vector>

// dp[S][j] = cheapest path starting at 0, visiting exactly the set
S, ending at j.
long long heldKarpTour(const std::vector<std::vector<long long>>&
d) {
    const int n = (int)d.size();
    if (n <= 1) return 0;
    const long long INF = LLONG_MAX / 4;
    const int full = 1 << n;
    std::vector<std::vector<long long>> dp(full, std::vector<long
long>(n, INF));
    dp[1][0] = 0;                                   // start at city
0
    for (int S = 1; S < full; ++S) {
        if (!(S & 1)) continue;                     // every state
contains city 0
        for (int j = 0; j < n; ++j) {
            if (dp[S][j] >= INF || !(S >> j & 1)) continue;
            for (int k = 1; k < n; ++k) {
                if (S >> k & 1) continue;
                const int T = S | (1 << k);
                dp[T][k] = std::min(dp[T][k], dp[S][j] + d[j][k]);
            }
        }
    }
    long long best = INF;
    for (int j = 1; j < n; ++j)
        if (dp[full - 1][j] < INF) best = std::min(best, dp[full -
1][j] + d[j][0]);
    return best;
}
```

Note the evaluation order: iterating `S` in increasing integer order works because `S ⊂ T ⟹ S < T` as integers. **Subset-lattice order is a topological order of the subproblem graph** — a small fact that makes every bitmask DP loop trivially correct.

Verified: matches brute-force enumeration of all `(n−1)!` permutations for `n` up to 8, across 40 random symmetric distance matrices.

3.11 Two more shapes worth having in hand

Longest weighted path in a DAG (CLRS Problem 14-1). The longest-simple-path problem is NP-complete on general graphs but **easy on a DAG** — because a DAG *supplies the ordering that DP requires*. Topologically sort, then relax edges in that order. `Θ(V + E)`.

Seam carving (CLRS Problem 14-8). Remove one pixel per row such that consecutive removed pixels are vertically or diagonally adjacent, minimizing total disruption. The number of seams is exponential in `m`; the DP is `O(mn)` with a rolling row. *(Avidan and Shamir's demo video is genuinely worth watching.)*

```
#include <algorithm>
#include <climits>
#include <utility>
#include <vector>

// Longest weighted path in a DAG: the topological order IS the
evaluation order.
long long dagLongestPath(int n, const
std::vector<std::vector<std::pair<int, long long>>>& adj,
                         int s, int t) {
    std::vector<int> indeg(n, 0);
    for (int u = 0; u < n; ++u) for (const auto& e : adj[u])
++indeg[e.first];
    std::vector<int> order;
    std::vector<int> stack;
    for (int u = 0; u < n; ++u) if (indeg[u] == 0)
stack.push_back(u);
    while (!stack.empty()) {
        const int u = stack.back(); stack.pop_back();
        order.push_back(u);
        for (const auto& e : adj[u]) if (--indeg[e.first] == 0)
stack.push_back(e.first);
    }
    const long long NEG = LLONG_MIN / 4;
```

```
    std::vector<long long> best(n, NEG);
    best[s] = 0;
    for (int u : order)
        if (best[u] > NEG)
            for (const auto& e : adj[u])
                best[e.first] = std::max(best[e.first], best[u] +
e.second);
    return best[t];
}

// Seam carving: minimum-disruption top-to-bottom seam, O(mn) time,
O(n) space.
long long minSeamCost(const std::vector<std::vector<long long>>& d)
{
    const int m = (int)d.size(), n = (int)d[0].size();
    std::vector<long long> prev(d[0]), cur(n);
    for (int i = 1; i < m; ++i) {
        for (int j = 0; j < n; ++j) {
            long long best = prev[j];
            if (j > 0)     best = std::min(best, prev[j - 1]);
            if (j + 1 < n) best = std::min(best, prev[j + 1]);
            cur[j] = d[i][j] + best;
        }
        prev.swap(cur);
    }
    return *std::min_element(prev.begin(), prev.end());
}
```

Verified: 100 random DAGs and 100 random grids each match brute force.

Part 4 — War Stories

War Story: Text Compression for Bar Codes (Skiena 10.4)

Symbol Technologies' **PDF-417** two-dimensional bar code stores ~1 000 bytes per 1-inch label. It encodes characters in **four modes** (uppercase, lowercase, digits, punctuation), 5 bits per character *within* a mode, plus 5 bits for a **mode shift** (next character only) or **mode latch** (permanent). Many encodings of the same text exist; the company used a **greedy** encoder ("look a few characters ahead and then decide which mode we would be

best off in — it works fairly well").

Skiena's question — *"How do you know it works fairly well? There might be significantly better encodings that you are simply not finding"* — got the answer *"I guess I don't know. But it's probably NP-complete to find the optimal coding."*

It isn't. *"From any given position in the text, we can either output the next character code (assuming it is available in our current mode) or decide to shift. As we moved from left to right through the text, our current state would be completely reflected by our current character position and current mode."* That sentence **is** the principle of optimality, discovered live:

```
M[i, j] = min_{1 ≤ m ≤ 4} ( M[i−1, m] + c(Sᵢ, m, j) )
```

where `c(Sᵢ, m, j)` is the cost of encoding character `Sᵢ` and switching from mode `m` to mode `j`. Only `4n` cells, `O(1)` each ⟹ **linear time**.

The measured result — and this is the number to quote when someone asks whether optimality is worth the trouble: on **13 000 real labels**, the DP encoder gave an **8% tighter encoding on average**, never worse than the greedy encoder, sometimes much better. In a medium where total capacity is a few hundred bytes, 8% is enormous. The DP was slightly slower to run, but *"this was not significant, because the bottleneck would be the time needed to print the label."*

> *"Our observed impact of replacing a heuristic solution with the global optimum is probably typical of most applications. Unless you really botch up your heuristic, you should get a decent solution. Replacing it with an optimal result, however, usually gives a modest but noticeable improvement, which can have pleasing consequences for your application."*

War Story: The Balance of Power (Skiena 10.6)

Three-phase AC power works best when loads on phases A, B, C are balanced. Given measured loads, assign each to a phase to balance them.

Step 1 — identify the problem. It's integer partition (subset sum with `k = Σsᵢ/2`) generalized to three parts. That's **NP-complete**, and Skiena said so. They got up to leave.

Step 2 — remember the pseudo-polynomial DP. Define `C[n, w_A, w_B]` = true if the first `n` loads can be partitioned with weight `w_A` on phase A and `w_B` on B. *(No need to track `w_C` — it's `Σsᵢ − w_A − w_B`. **Dropping a redundant dimension is a standard state-space reduction; look for it every time.**)*

```
C[n, w_A, w_B] = C[n−1, w_A − sₙ, w_B] ∨ C[n−1, w_A, w_B − sₙ] ∨
C[n−1, w_A, w_B]
```

`nk²` cells, `O(1)` each ⟹ **`O(nk²)`**.

Step 3 — the part that shows DP's real value: the objective kept changing, and the recurrence didn't.

- *"It is not a cost-free operation to change which phase a load is on"* — they wanted the most balanced assignment **minimizing the number of changes**. Same recurrence, storing a **cost** instead of a **flag**:

  ```
  C[n, w_A, w_B] = min( C[n−1, w_A − sₙ, w_B] + 1,  C[n−1,
  w_A, w_B − sₙ] + 1,  C[n−1, w_A, w_B] )
  ```

- Then they wanted a solution that *never got seriously unbalanced at any point along the line* (a globally balanced solution might load all of A first, which is bad in practice). **Same recurrence again** — just set `C[n, w_A, w_B] = ∞` whenever the state is deemed too unbalanced.

> *"That is the power of dynamic programming. Once you can reduce your state space to a small enough size, you can optimize just about anything. Just walk through each possible state and score it appropriately."*

And the practical escape hatch: the `O(nk²)` runtime grows quadratically in the load range, which could be a problem — but *"binning the loads by (say) `sᵢ/10` would reduce the running time by a factor of 100 and produce solutions that were still pretty good."* **Rounding the state space is how a pseudo-polynomial DP becomes an approximation scheme** (M20).

Part 5 — The Design Checklist

When you suspect DP, work these in order:

1. **Is it an optimization (or counting, or feasibility) problem over a set of choices?** If it's "find any solution", consider greedy or search instead.
2. **Is there a natural left-to-right order?** Strings, sequences, intervals, rooted trees, points on a polygon, prefixes of an integer. **No order ⟹ probably exponential.**

3. **Ask the generative question:** *what would I need to know about the first* `n−1` *elements to decide what to do with the* `n`*-th?* The answer is your state.
4. **Write the recurrence.** Include the **boundary conditions** — resolve the smallest value of every argument.
5. **Count the distinct states.** Polynomial ⟹ proceed. Exponential ⟹ look for a redundant dimension to drop, or accept 2^n.
6. **Verify optimal substructure by cut-and-paste**, and check that the subproblems are **independent** (don't share a resource).
7. **Pick an evaluation order** — any reverse topological order of the subproblem graph. In practice: by length, by index, by subset size, by topological order of a DAG.
8. **Compute the cost:** states × transitions per state.
9. **Decide memoization vs. tabulation.** Memoize first if you're under time pressure.
10. **Add the choice table** if you need the actual solution, not just its value.
11. **Optimize space** only after it's correct, and only if reconstruction doesn't need the full table.

Recognition patterns

SIGNAL	STATE SHAPE	EXAMPLE
"subsequence of / alignment of two strings"	`dp[i][j]` over prefixes	LCS, edit distance
"partition a sequence into `k` contiguous parts"	`dp[i][k]`	linear partition, printing neatly
"optimally parenthesize / split a range"	`dp[i][j]` over intervals, loop by length	matrix chain, optimal BST, CYK, polygon triangulation
"choose a subset subject to a numeric budget"	`dp[i][budget]`	knapsack, subset sum, coin change
"path through a grid with limited moves"	`dp[i][j]`	seam carving, edit distance (again)
"longest/most in a sequence ending at `i`"	`dp[i]`	LIS, max subarray, house robber
"the last decision was `x`"	`dp[i][last]`	bar-code modes, stock-trading states, Viterbi
"visit all of a small set"	`dp[mask][last]`	Held-Karp, assignment problems
"on a tree, take-or-skip this node"	`dp[v][0/1]`	company party (Problem 14-6), tree independent set
"a DAG is given or implied"	`dp[v]` in topological order	DAG longest path, all of the above really

The unifying view worth stating out loud: *every* DP is a shortest/longest path computation on the subproblem DAG. Nodes are states, edges are transitions, edge weights are the cost attributable to the choice. That's why topological order is always the right evaluation order, and why "no order ⟹ no DP" is the fundamental limitation.

The CLRS problem set — a map of what DP can do

PROBLEM	SHAPE	WHY IT'S WORTH KNOWING
14-1 Longest simple path in a DAG	`dp[v]` in topological order	the acyclic case of an NP-complete problem
14-2 Longest palindrome subsequence	interval, or `LCS(s, reverse(s))`	two derivations, both instructive
14-3 Bitonic euclidean TSP	`dp[i][j]` over two path ends	$O(n^2)$ for a restricted TSP
14-4 Printing neatly	`dp[i]` + line-cost	this is TeX's paragraph breaker
14-5 Edit distance (6 operations)	`dp[i][j]`	generalized alignment incl. twiddle and kill
14-6 Planning a company party	`dp[v][take?]` on a tree	the canonical tree DP
14-7 Viterbi	`dp[v][k]` with `(max, ×)`	speech recognition; HMM decoding
14-8 Seam carving	grid `dp`	image resizing, and a great demo
14-9 Breaking a string	interval `dp`	matrix chain in disguise
14-10 Investment strategy	`dp[year][investment]`	note part (d): a cap **destroys** optimal substructure
14-11 Inventory planning	`dp[month][inventory]`	production planning; pseudo-polynomial in `D`
14-12 Free-agent baseball players	`dp[position][budget]`	knapsack with groups

Common Mistakes

MISTAKE	CONSEQUENCE
Building the table before writing the recurrence	You will fill cells with the wrong thing. **Recurrence first, always.**
A state that doesn't capture what the next decision depends on	Wrong answers. (Skiena's "is the LIS 5 or 6?" is the model failure.)
Constraining the subproblem space too narrowly (`A₁⋯A_j` instead of `A_i⋯A_j`)	The recurrence can't close
Forgetting boundary conditions	Off-by-one or garbage at the edges — the #1 source of DP bugs
Assuming optimal substructure without checking independence	Longest simple path; rod cutting with piece limits; investment with a cap
Memoizing a divide-and-conquer algorithm	No speedup — the subproblems never repeat
Iterating the knapsack capacity **upward** in the 1-D version	Silently solves *unbounded* knapsack instead
`lower_bound` vs `upper_bound` in `O(n lg n)` LIS	Strictly-increasing vs non-decreasing, silently
Space-optimizing to two rows and then trying to reconstruct	The path information is gone; use Hirschberg or keep the table
Calling an `O(nk)` subset-sum algorithm "polynomial"	It is **pseudo-polynomial** — exponential in the input's bit length
Recomputing a range sum inside the innermost loop	An extra factor of `n`; **prefix sums** fix it
Deep recursion in a memoized solution on large inputs	Stack overflow; convert to bottom-up
Not verifying the reconstruction	The value can be right while the traceback is wrong; **always check that the reconstructed solution actually achieves the reported cost**

Complexity Summary

PROBLEM	STATES	CHOICES	TIME	SPACE	SPACE IF VALUE ONLY
Fibonacci	n	1	$\Theta(n)$	$\Theta(n)$	$\Theta(1)$
Rod cutting	n	n	$\Theta(n^2)$	$\Theta(n)$	$\Theta(n)$
Matrix chain	n^2	n	$\Theta(n^3)$	$\Theta(n^2)$	$\Theta(n^2)$
LCS	mn	$O(1)$	$\Theta(mn)$	$\Theta(mn)$	$\Theta(\min(m,n))$
Edit distance	mn	3	$\Theta(mn)$	$\Theta(mn)$	$\Theta(\min(m,n))$
LIS (quadratic)	n	n	$\Theta(n^2)$	$\Theta(n)$	$\Theta(n)$
LIS (patience)	—	—	$\Theta(n \lg n)$	$\Theta(n)$	$\Theta(n)$
Subset sum	nk	2	$\Theta(nk)$ **pseudo-poly**	$\Theta(nk)$	$\Theta(k)$
0-1 knapsack	$n \cdot C$	2	$\Theta(nC)$ **pseudo-poly**	$\Theta(nC)$	$\Theta(C)$
Ordered partition	nk	n	$\Theta(kn^2)$	$\Theta(nk)$	$\Theta(nk)$
CYK parsing	$n^2 \cdot \lvert G \rvert$	n	$\Theta(n^3)$	$\Theta(n^2)$	$\Theta(n^2)$
Optimal BST	n^2	n	$\Theta(n^3) \to \Theta(n^2)$ (Knuth)	$\Theta(n^2)$	$\Theta(n^2)$
Held-Karp TSP	$2^n \cdot n$	n	$\Theta(2^n n^2)$	$\Theta(2^n n)$	same
DAG longest path	V	out-degree	$\Theta(V + E)$	$\Theta(V)$	$\Theta(V)$
Seam carving	mn	3	$\Theta(mn)$	$\Theta(n)$	$\Theta(n)$

One-Page Recall

- **DP = exhaustive search + memory.** Correctness from considering all possibilities; efficiency from never recomputing.
- **Recurrence first, table second.** Skiena: *"Only after we have a correct recursive algorithm can we worry about speeding it up by using a results matrix."*
- **Skiena's 3 steps:** (1) write the recurrence; (2) show the number of distinct parameter values is a small polynomial; (3) specify an evaluation order.
- **CLRS's 4 steps:** characterize optimal structure → define the value recursively → compute bottom-up → construct the solution.

- **The generative question:** *what would I need to know about the first `n−1` elements to decide what to do with the `n`-th?* That is your state.
- **Two hallmarks: optimal substructure** (verify by cut-and-paste) and **overlapping subproblems** (a polynomial number of distinct states). They are compatible: *independent* = "don't share resources"; *overlapping* = "the same subproblem shows up under different parents".
- **Time ≈ states × choices per state**, or `Θ(V + E)` of the subproblem graph.
- **Evaluation order = reverse topological sort** of the subproblem graph. Memoization = DFS of it. If the graph has a cycle, no DP exists.
- **DP fails two ways:** the recurrence is wrong (doesn't enforce a constraint, like simplicity), or there's no ordering (state space blows up).
- **Left-to-right objects are DP's home:** strings, permutations, points around a polygon, leaves of a search tree. Without an inherent order, expect exponential.
- **Memoize when the state space is sparse or the order is awkward; tabulate when you need constant factors or rolling-array space.**
- **Reconstruct** with a parent/choice table; recursing before emitting reverses the traceback for free.
- **Edit distance is the master recurrence.** Change the boundary row to 0 ⟹ approximate substring search. Make substitution expensive ⟹ LCS. LCS against the sorted input ⟹ LIS.
- **Prefix sums** turn `O(n)` range costs into `O(1)`, dropping a factor of `n`. Look for this in every interval DP.
- **Swap the semiring** to change the question: `(∨,∧)` feasibility, `(min,+)` optimization, `(+,×)` counting, `(max,×)` most-probable.
- **Pseudo-polynomial ≠ polynomial.** `O(nk)` for subset sum is exponential in the `O(log k)` bits describing `k`. Multiplying every input by `10⁶` slows it down `10⁶×`.
- **`O(2ⁿ)` beats `O(n!)` by enough to matter** — Held-Karp handles `n ≈ 25`.
- **Skiena's closing pitch:** *"Once you can reduce your state space to a small enough size, you can optimize just about anything. Just walk through each possible state and score it appropriately."*

Next: M12 — Greedy Algorithms (CLRS 15 + Skiena) — the greedy-choice property, matroids, Huffman coding, and the discipline of proving that the locally best choice is globally safe.